

FortLite

Five-Stage Rebuild

A controlled path from the restored 3D prototype to a polished, original browser battle royale with real combat, building, bots, and join-code multiplayer.



VERSION	DELIVERY MODEL	PRIMARY TARGET
1.0 / Sept 2026	One playable build per stage	Desktop Chrome

The build contract

FortLite will stay a third-person 3D battle royale. Every stage ends in a build that can be played and judged in Chrome. The next stage starts only after the current exit gate passes.

01	Third-person from drop to victory	Parachute, responsive movement, readable camera, real weapons and inventory.
02	A complete original island	Distinct locations, useful buildings, clear routes, loot density and storm pacing.
03	Opponents that create real fights	Bots loot, rotate, aim, take cover, build, pressure and recover at varied skill levels.
04	Real multiplayer	Join codes backed by an authoritative server; no tab-only or cosmetic party system.
05	Chrome-ready finish	Stable frame pacing, scalable quality, working menus and a repeatable full-match test.

Scope allocation

15%	20%	25%	25%	15%
Core	World	Combat + AI	Multiplayer	Polish

Rules that protect quality

- Use the restored Three.js game as the starting foundation; the rejected top-down Harbor Run build stays removed.
- Keep the FortLite name, parachute, Rifle / Shotgun / SMG slots, and the established keyboard controls.
- Use original map geometry, art, sounds, interface, and identity rather than copying Fortnite assets or exact layouts.
- Treat compilation as a basic check. Each gate also requires an actual Chrome playthrough and a visual comparison against the approved concept direction.
- Do not claim multiplayer until two independent browser clients can join by code and remain synchronized through a fight.

Restore the real game and lock the controls

Deliver one clean solo match loop on the existing 3D foundation. The player can drop, move, loot, switch weapons, shoot, build simple cover, survive the storm, and finish a match.

PLAYER-FACING PROOF

A new player can complete a full solo match without a broken control, dead-end screen, or fake placeholder mechanic.

Build package

- Delete the rejected Harbor Run implementation and keep FortLite as a full-screen Three.js experience.
- Stabilize third-person movement, sprint, jump, collision, camera orbit, aim mode and pointer lock.
- Make keys 1, 2 and 3 switch Rifle, Shotgun and SMG immediately, with visible slot state and held-weapon feedback.
- Repair firing, reload, ammo use, pickups, damage, elimination, storm damage and match reset.
- Keep Q / Z / X / C / R building inputs working with readable placement previews and basic collision.

Verification

- Start, pause, resume, lose, win and restart from the UI.
- Drop in five different directions without clipping through the island.
- Cycle all weapons while walking, sprinting, aiming and out of ammo.
- Build wall, floor and ramp on valid terrain; reject invalid placement cleanly.
- Run at least one complete bot match with zero console errors.

EXIT GATE

Ship the stage only when the complete solo loop works in desktop Chrome at a stable playable frame rate and every listed key produces visible, correct behavior.

Deferred until later stages

Final art, advanced bot tactics, internet multiplayer, detailed audio mixing and exhaustive performance tuning remain outside this gate. Stage 1 proves the game itself is real and controllable.

Build the island players want to explore

Replace prototype-looking terrain and structures with a coherent coastal island: readable from the air, varied on foot, and designed around combat routes rather than decoration alone.

PLAYER-FACING PROOF

The first drop presents several obvious choices, and every named area supports looting, cover, traversal and a memorable visual landmark.

World and art

- Sculpt coastlines, cliffs, beaches, hills, roads, tree lines, water and a clear playable boundary.
- Create three anchor locations: harbor district, hill settlement and lighthouse overlook, plus smaller compounds between them.
- Rebuild houses, warehouses, docks, towers and interiors with doors, stairs, windows, roofs and believable scale.
- Add bright sky, directional sunlight, soft shadows, atmospheric haze, water shading and a restrained stylized material palette.
- Improve player, glider, weapon and traversal animation silhouettes so actions read at combat distance.

Layout and presentation

- Place loot by risk and route value; high-tier areas expose the player to stronger sightlines.
- Add cover rhythm across open spaces without turning the island into visual clutter.
- Redesign HUD to match the concept: health top-left, status top-right, minimap bottom-left and slots bottom-right.
- Use compact menus and legible high-contrast text at 1280x720 through 1920x1080.
- Add landing, pickup, hit, elimination, storm and build effects that communicate state quickly.

EXIT GATE

Ship the stage only after each anchor location passes an air-view, street-level and interior screenshot review, and a player can navigate between all locations without getting stuck.

Performance budget

Favor instancing, shared materials, simple collision meshes, object pooling and distance-based detail. Visual upgrades must keep Stage 1 controls and match flow responsive.

Make combat, building and bots fun

Turn the island into a battle royale rather than a shooting sandbox. Weapons need distinct roles, building must save lives, and bots must create readable pressure at several skill levels.

PLAYER-FACING PROOF

A ten-minute session produces different fights, useful build decisions and believable opponents instead of stationary targets or perfect aim machines.

Combat and building

- Tune Rifle for dependable mid-range fire, Shotgun for close burst damage and SMG for short-range pressure.
- Add spread, recoil, falloff, reload timing, muzzle flash, tracers, impacts, hit markers, directional damage and strong audio cues.
- Make weapon switching buffered and reliable; show ammo and active slot without covering the action.
- Improve build snapping, placement range, material costs, damage states and destruction feedback.
- Keep building simple: wall, floor and ramp are enough if each works during an actual fight.

Bot behavior

- Use perception, memory and finite states for looting, rotating, engaging, retreating, healing and harvesting.
- Create skill profiles through reaction time, aim error, burst length, aggression, awareness and build frequency.
- Use navigation routes and local obstacle avoidance so bots enter buildings, cross terrain and escape the storm.
- Let bots choose cover and build one or two defensive pieces under pressure.
- Prevent unfair knowledge: no shooting through walls, instant 180-degree snaps or tracking unseen players.

EXIT GATE

Ship the stage after repeated matches show weapon-role balance, no broken bot states, varied winners, and opponents that can loot, rotate, fight and respond to player builds.

Test evidence

Record bot accuracy, average fight duration, weapon pick rate, elimination cause and final-circle survival. Use those numbers with playtests to tune behavior instead of guessing.

Add real join-code multiplayer

Build multiplayer as server-backed gameplay. A room code must connect separate browsers, and the server must own match truth for players, shots, loot, builds, bots and the storm.

PLAYER-FACING PROOF

Two independent Chrome clients join the same code, see the same match, damage each other, build, eliminate, spectate and recover from a reconnect.

Networking architecture

- Add a dedicated authoritative match server with room creation, six-character join codes and lobby lifecycle.
- Send input to the server; broadcast snapshots for movement, aim, animation, weapon state, health and builds.
- Use interpolation for remote players, prediction for local movement and server reconciliation for corrections.
- Validate fire rate, ammo, damage, placement, inventory and storm state on the server.
- Provide heartbeat, reconnect token, host-independent rooms and clear disconnect handling.

Match flow

- Create room, copy code, join room, ready up, launch, play, spectate and rematch.
- Support human players plus server-controlled bots so low-population rooms still start quickly.
- Synchronize the drop, loot ownership, inventory changes, eliminations, builds and shrinking safe zone.
- Show latency and connection status only when useful; keep networking language out of the normal HUD.
- Define a deployment target for both the static client and persistent WebSocket service before calling the stage complete.

EXIT GATE

Ship the stage only after cross-device testing proves room codes work outside one browser, a 20-minute soak test stays synchronized, and reconnects do not duplicate players or loot.

Largest project risk

This stage needs hosted server infrastructure, logging and load tests. It cannot be replaced by BroadcastChannel, localStorage or two tabs that only appear connected.

Optimize, polish and release

Finish FortLite as a dependable ChudGames title. This stage removes rough edges, tunes the whole experience, proves performance and verifies every path a player can click.

PLAYER-FACING PROOF	A first-time player can enter a match quickly, understand the interface, finish or leave cleanly, and immediately play again without refreshes or broken state.
---------------------	---

Performance and stability

- Profile CPU, GPU, memory, draw calls, triangles, network traffic and garbage collection in real matches.
- Target 60 FPS at 1080p on a recommended desktop; provide scalable settings for weaker machines.
- Use dynamic resolution, shadow tiers, distance detail, instancing, pooling and capped particles where measurements justify them.
- Keep input and simulation stable when rendering slows; stop all loops, listeners and Three.js resources on exit.
- Test long sessions, tab focus changes, resize, fullscreen, disconnects and repeated rematches.

Release polish

- Finish lobby, loading, pause, settings, defeat, victory, spectate and rematch transitions.
- Verify every button, keybind, weapon slot, build selection, join-code error and return-to-library action.
- Mix footsteps, weapons, impacts, storm, builds and ambience so nearby danger is readable without visual UI.
- Add a short first-match control guide and settings persistence without slowing entry into play.
- Run build, tests, console review, visual QA and a full two-client release checklist before pushing main.

EXIT GATE	Release only when no blocker or high-severity defects remain, multiplayer passes independent-client testing, and measured Chrome performance meets the agreed hardware targets.
-----------	---

Definition of done

Complete match loop	Concept-level hierarchy	Real room codes	Stable Chrome frame pacing
All controls and buttons	Original island identity	Server-owned outcomes	Clean repeated sessions

Recommended execution: finish and approve Stage 1 before increasing art scope. The strongest first checkpoint is a plain-looking game that already feels correct to control, fight and finish.